Neural Computing Lecture Notes

Elena Raponi

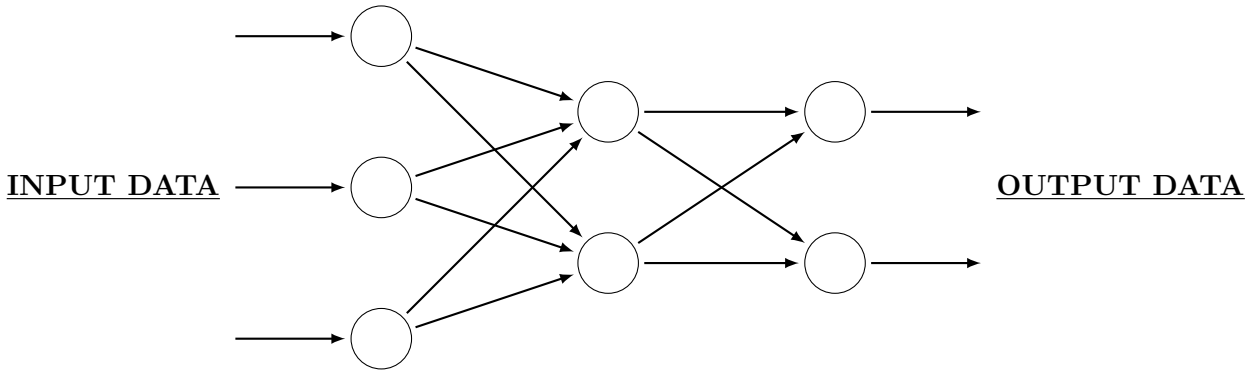
March 3, 2026

Contents

1 Introduction

1.1 What is a neural network?

We have DATA and want to figure out RULES, so that we can make correct predictions to new data!



Idea of a Neural Network

We begin with the idea of what is a NEURON!

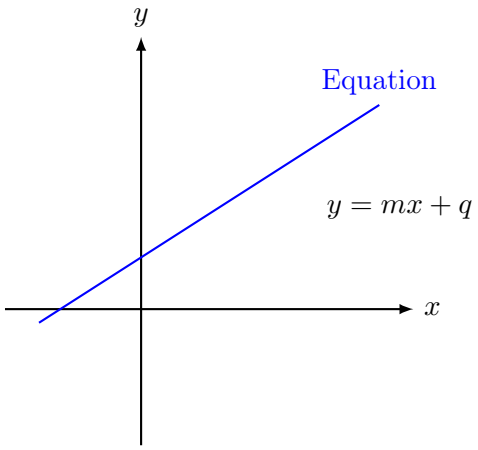
$$\begin{bmatrix}1\\0\end{bmatrix} \text{ or } \begin{bmatrix}0\\1\end{bmatrix}$$

"cat" "dog"

One-Hot Encoding Example: Upper output neuron represents a cat, while the lower neuron represents a dog.

1.2 ML Paradigm

Consider simple functions in mathematics:



Equation of a Line

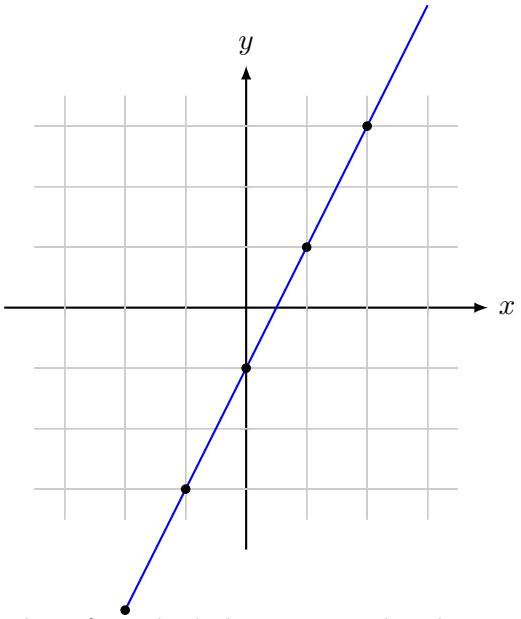
$$y = mx + q \tag{1}$$

Slope m Y-Intercept q

New Notation

$$y = Wx + B \tag{2}$$

Weight W Bias B



Let's consider that we know the equation
$y = 2x - 1 \quad W = 2 \quad B = -1$
We can see that

$$\begin{array}{ccc} x = 0 & \Rightarrow & y = -1 \\ x = 1 & \Rightarrow & y = 1 \\ x = 2 & \Rightarrow & y = 3 \\ \uparrow & & \uparrow \\ \text{We know} & & \text{We compute} \end{array}$$

(But we know the values of the other parameters W, B)

What if we don't know W and B but we have input/output data? $\longrightarrow$ We need to figure out the rule!

ML-PARADIGM:

x	y	$\hat{y}_1$	$\hat{y}_2$	$\hat{y}_3$
-4	-9			
-3	-7			
-2	-5	-5	-3	-4
-1	-3			
0	-1	1	1	0
1	1			
2	3			
3	5	10	7	6
4	7			
		LOSS 7	LOSS 6 Better	LOSS 3 Better!!!

ANSWER: $y = 2x - 1$
We know the general formula

$y = Wx + B$

and we start with a guess

GUESS N°1: $y = 3x + 1$

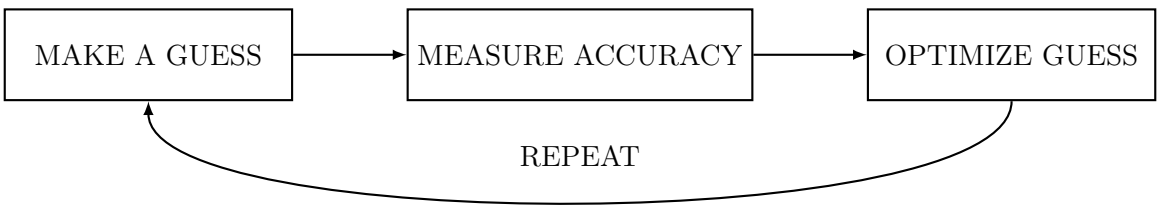
We can compute the answers for that guess ($\hat{y}$) and compare them with the real answers!

GUESS N°2: $y = 2x + 1$

GUESS N°3: $y = 2x + 0$

$\Rightarrow$ We become better every time we guess!

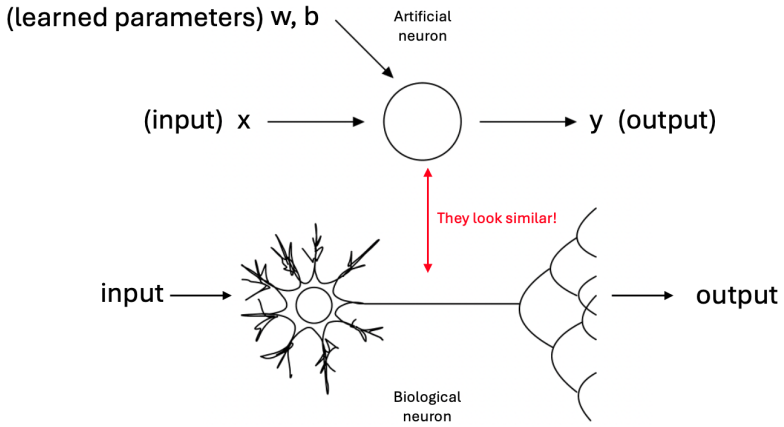
$\Leftrightarrow$ COST/LOSS REDUCED (i.e., we give better answers any time we guess)



1.3 Connection to the Biological Neuron

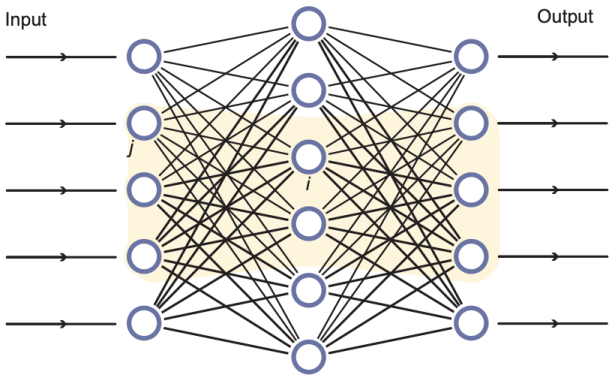
The process is often shown like this

We can get the internal parameters fit so that, given certain input data, we can get the desired output!



Comparison of Neural Network and Biological Neuron

Neurons can be networked together to learn more complex patterns ANN → emulator of our brain (structure of interconnected neurons)



Schematic sketch of a feed-forward neural network.

- Neurons are gathered into layers, and each layer accepts inputs only from the previous layer, and the signal only passes to neurons of the next layer!
- Each neuron is just a mathematical function that can learn parameters such that the NN can get the desired output for a set of desired inputs.

2 The Hopfield Network

An associative memory model using the Hebbian rule (*neurons that fire together, wire together*) for all possible pairs (i, j) with binary units and asynchronous updating. Syncronous updating is also possible but it would be less stable. The asynchronous updates ensure stability because of the monotonic changes in the energy function.

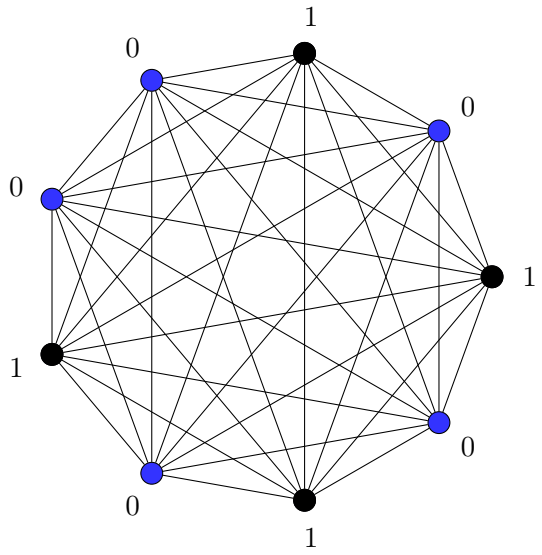
2.1 Associative Memory Problem

Store a set of p patterns $\vec{\xi}^1, \ldots, \vec{\xi}^p$, with $\vec{\xi}^i_j = \{\xi^i_1, \ldots, \xi^i_N\}$ so that, when presented with a new pattern $\vec{x}$, the network responds by producing whichever one of the stored patterns that most closely resembles $\vec{x}$

Patterns can be visualized in different ways:

$\vec{\xi}^i$= STORED PATTERN = [010101010]

$$\xi^i = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} =$$



Evolution of a tested pattern:

$$\vec{x}_i = \text{TEST PATTERN} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$\downarrow$$

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$\downarrow$$

$$\begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$\downarrow$$

$$\begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \end{bmatrix} = \vec{\xi^i}$$

Weights evolve and restore the $\vec{\xi^i}$ pattern.

2.2 Hamming Distance

If we have p stored pattern, the idea is that the weights evolve in such a way that the network reproduces the pattern ξ^p with minimum **Hamming Distance** from the test pattern $\vec{x}$

$$d(\vec{x}, \vec{\xi}) = \sum_{i=1}^{N} [\xi_i(1 - x_i) + (1 - \xi_i)x_i] \tag{3}$$

Example

$$\vec{x^i} = \begin{bmatrix} 0 & 0 & 0 \,|\, 0 & 0 & 0 \,|\, 0 & 1 & 0 \end{bmatrix}$$
$$\vec{\xi} = \begin{bmatrix} 0 & 1 & 0 \,|\, 1 & 0 & 1 \,|\, 0 & 1 & 0 \end{bmatrix}$$

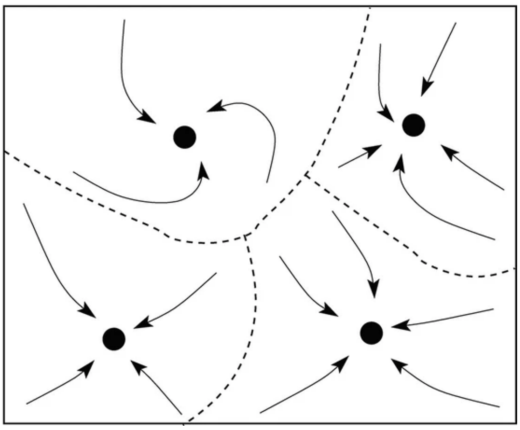
$$\begin{aligned} d &= 0\cdot(1-0)+(1-0)\cdot 0+1\cdot(1-0)+(1-1)\cdot 0+\cdots \\ &= 0+1+0+1+0+1+0+0+0 \\ &= 3 \end{aligned}$$

2.3 Notation

For mathematical convenience, rather than the binary states $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$, we will use the following notation:

FIRING = unit value +1
NOT FIRING = unit value -1

Node state of test pattern: $x_i = O_i(0)$
Node state of stored pattern: ξ_i
Node state of evolving pattern at time t: $O_i(t)$



Configuration Space: a set of attractors states (black circles) and their corresponding basins of attraction separated by dashed lines. The curved arrows represent possible trajectories to reach the attractors from some given initial conditions.

Dynamics of the network represented by this model:

$$O_i(t+1) = g\left(\sum_T w_{ij}O_j(t) + b_i\right) \tag{4}$$

$$\rightsquigarrow O_i(t+1) = \text{sgn}\left(\sum_j w_{ij}O_j(t)\right)$$

$w_{ij} \propto \xi_i\xi_j$
$\vec{\xi}^1 \dots \vec{\xi}^p$ = stored patterns
$\vec{x}$ = arbitrary input pattern
$O(0) = \vec{x}$
N = number of units/nodes in the HN

Goal: to design a network that evolves so that is approaches any desired pattern $\vec{\xi} = \{\xi_1, \dots, \xi_N\}$.

2.4 Single Stored Pattern

2.4.1 Stability Condition

Definition. Let's consider one stored pattern $\vec{\xi}$. The condition for $\vec{\xi}$ to be a stable pattern (attractor) is

$$O_i = \text{sgn}\left[\sum_{j=1}^{N} w_{ij}\xi_j\right] = \xi_i \tag{5}$$

Statement. We obtain this if $w_{ij} = w_{ji} = \alpha\xi_i\xi_j$, $\alpha > 0 \quad (w_{ij} \propto \xi_i\xi_j)$
Proof. Let $W = \alpha\xi_i\xi_j$ be the weight matrix:

$$O_i = \text{sgn}[\sum_j w_{ij}\xi_j] = \text{sgn}[\sum_j \propto \xi_i\xi_j\xi_j] = \text{sgn}\left[\alpha\xi_i\sum_{j=1}^{N} 1\right] = \text{sgn}(\propto N\xi_i) = \text{sgn}(\xi_i) = \xi_i$$

$\Longrightarrow O_i = \xi_i$ STABLE! $\Longrightarrow$ We can take $\alpha = \frac{1}{N}$ when $N = \#$ nodes $\longrightarrow$ $w_{ij} = \frac{1}{N}\xi_i\xi_j$

2.4.2 Example

Design a network of 3 neurons that resembles a pattern (1,1,-1)

$$W = \frac{1}{N}\vec{\xi}\vec{\xi}^{\top} = \frac{1}{3}\begin{pmatrix} 1 \\ 1 \\ -1 \end{pmatrix}\begin{pmatrix} 1 & 1 & -1 \end{pmatrix} = \frac{1}{3}\begin{bmatrix} 1 & 1 & -1 \\ 1 & 1 & -1 \\ -1 & -1 & 1 \end{bmatrix}$$

1. Is $\vec{\xi}$ stable?

$$\vec{0} = \operatorname{sgn}(W \cdot \vec{\xi}) = \operatorname{sgn}\left[\frac{1}{3}\begin{pmatrix} 1 & 1 & -1 \\ 1 & 1 & -1 \\ -1 & -1 & 1 \end{pmatrix}\begin{pmatrix} 1 \\ 1 \\ -1 \end{pmatrix}\right] = \operatorname{sgn}\left[\frac{1}{3}\begin{pmatrix} 3 \\ 3 \\ -3 \end{pmatrix}\right]$$
$$= \operatorname{sgn}\begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix} = \vec{\xi}$$

$\Longrightarrow$ we got Equation 5 $\Longrightarrow$ the answer is YES!

2. Is $\vec{\xi}$ and attractor? Let's try a test pattern $\vec{x} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$

Let's apply Equation 4:

$$\vec{O} = \operatorname{sgn}\left[\frac{1}{3}\begin{pmatrix} 1 & 1 & -1 \\ 1 & 1 & -1 \\ -1 & -1 & 1 \end{pmatrix}\begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}\right] = \operatorname{sgn}\left[\frac{1}{3}\begin{pmatrix} 1 \\ 1 \\ -1 \end{pmatrix}\right] = \operatorname{sgn}\begin{bmatrix} 1/3 \\ 1/3 \\ -1/3 \end{bmatrix} = \vec{x}$$

$\longrightarrow \vec{\xi} =$ attractor state

2.4.3 Attractors

Statement: $\vec{\xi}$ is an attractor of the network defined by the weights $w_{ij} = \frac{1}{N}\xi_i\xi_j$

Proof:

Let's consider the total net input $h_i = \sum_{J=1}^{N} w_{ij}x_j$

$$\Rightarrow h_i = \sum_j \frac{1}{N}\xi_i\xi_j x_j = \frac{1}{N}\xi_i \sum_j \xi_j x_j = \frac{1}{N}\xi_j\left[\sum_{j=\text{correct}} \xi_j x_j + \sum_{j=\text{wrong}} \xi_j x_j\right] = \frac{\xi_i}{N}(N_{\text{correct}} - N_{\text{wrong}})$$

$$\Rightarrow O_i = \operatorname{sgn}(h_i) = \operatorname{sgn}\left[\frac{\xi}{N}(N_{\text{correct}} - N_{\text{wrong}})\right] = \begin{cases} \xi_i & \text{if more than or exactly } \frac{N}{2} \text{ are correct,} \\ -\xi_i & \text{otherwise/opposite case.} \end{cases}$$

$\Rightarrow \xi_i$ is an attractor! AND in this simple case we also have another attractor at the reversed state $-\xi$.

2.5 Multiple Stored Patterns

How do we get the system to restore the most similar pattern to the test del out of many patterns $\vec{\xi}^1, \ldots, \vec{\xi}^p$?

2.5.1 Hebbian Rule

To store multiple patterns, we make w_{ij} a **superposition** of terms:

$$w_{ij} = \frac{1}{N} \sum_{\mu=1}^{p} \xi_i^\mu \xi_j^\mu \tag{6}$$

$\Rightarrow$ the network states of $\vec{x}$ will evolve to restore the closest pattern $\vec{\xi}^\mu$. Equation 6 is called **Hebbian Rule** ("Networks which fire together, wire together").
Indeed, if ξ_i and ξ_j are active states in many patterns, their connection strength increases. And their activity becomes more correlated over time.
And the other way around: if the units fire asynchronously, their connection weight decreases. (This is the equivalent for the phenomenon of synoptic plasticity in biological NNs)

2.5.2 Stability Condition

The **Stability Condition** for a pattern ξ_i^ν is $O_i = \text{sgn}(h_i^\nu) = \xi_i^\nu, \quad \forall i = 1, \ldots, N.$

Proof: Let's consider the net input

$$\begin{aligned} h_i^\nu \equiv \sum_{j=1}^{N} w_{ij} \xi_j^\nu &= \frac{1}{N} \sum_{j=1}^{N} \sum_{\mu=1}^{p} \xi_i^\mu \xi_j^\mu \xi_j^\nu = \\ &= \frac{1}{N} (\sum_{j=1}^{N} \xi_i^\nu \underbrace{\xi_j^\nu \xi_j^\nu}_{=1} + \sum_{j=1}^{N} \sum_{\mu \neq \nu} \xi_i^\mu \xi_j^\mu \xi_j^\nu) = \\ &= \frac{1}{N} \underbrace{\sum_{j=1}^{N} \xi_i^\nu}_{N \xi_i^\nu} + \frac{1}{N} \sum_{j=1}^{N} \sum_{\mu \neq \nu} \xi_i^\mu \xi_j^\mu \nu_j^\nu = \\ &= \xi_i^\nu + \underbrace{\frac{1}{N} \sum_{j=1}^{N} \sum_{\mu \neq \nu} \xi_i^\mu \xi_j^\mu \xi_j^\nu}_{\equiv A}. \end{aligned}$$

Now, if $|A| < 1$, then $O_i = \text{sgn}(h_i^\nu) = \text{sgn}(\xi_i^\nu + A) = \text{sgn}(\xi_i^\nu) = \xi_i^\nu \quad \Rightarrow$ **Stability**.
A is named to as *crosstalk* term and refers to the undesired interactions between stored patterns, generating interference, and hence retrieval error.

In the chain of equations above we have used:

- First equivalence: definition of net input;
- Second equal sign: hebbian rule (Equation 6);
- Third equal sign: splitted the sum over the patterns into the one addition term we have for $\mu = \nu$ and the rest of the sum for $\mu \neq \nu$;

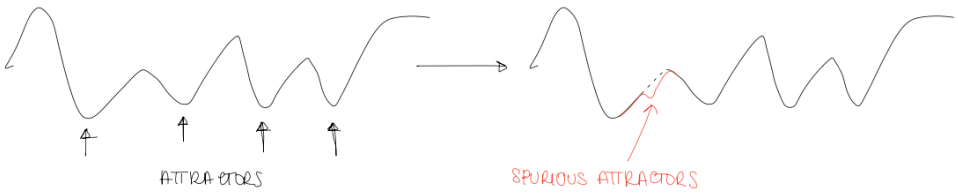
- Fourth equal sign: ξ_i^ν does not depend on j, which is the index of the sum. Thus, it is simply a constant repeated N times;
- Fifth equal sign: $\frac{1}{N}$ and N cancel out and we define the second addition term as the crosstalk term A.

2.5.3 Spurious Attractors

Let's consider the constant term $A = \frac{1}{N}\sum_j \sum_{\mu\neq\nu} \xi_i^\mu \xi_j^\mu \xi_j^\nu$.
It can also be seen as:

$$A = \frac{1}{N}\sum_{\mu\neq\nu} \xi_i^\mu \underbrace{\sum_j \xi_j^\mu \xi_j^\nu}_{\vec{\xi^\mu}\cdot\vec{\xi^\nu}}. \tag{7}$$

Here, $\vec{\xi^\mu}\cdot\vec{\xi^\nu}$ is the scalar or inner product of the two patterns ξ_j^μ and ξ_j^ν, which is equal to zero (best possible scenario, as the intereference would not happen and we would have $\xi_i^\nu + A = \xi_i^\nu \implies O_i = \xi_i^\nu$) if and only if the two vectors are orhogonal. Thus, if all the memories are orthogonal ($\vec{\xi^\mu}\cdot\vec{\xi^\nu} = 0$), then they are all stable attractors. If one of the memories (e.g., $\vec{\xi^1}$) is very close to $\vec{\xi^\nu}$, then $\vec{\xi^1}\cdot\vec{\xi^\nu} \approx N$ and $O_i = \text{sgn}(\xi_i^\nu + \xi_i^1) \neq \xi_i^\nu$, which means that ξ^ν is not an attractor.
However, patterns do not need to be necessarily orthogonal. It is sufficient that they are different enough so that $|A| < 1$. If the patterns are too similar, or if too many patterns are learned, the HN risks to converge to an in-between pattern, which is a combination of the learned patterns. These patterns are called **spurious attractors**, and correspond to unexpected minima on the energy landscape (see figure below).



2.5.4 Capacity

The *capacity* of a HN is defined as the maximum number of patters it can store before retrieval error occurs due to interference.
To understand what is the capacity of a HN, let's conveniently redefine the crosstalk term as:

$$C_i^\nu \equiv -\xi_i^\nu A = -\xi_i^\nu \frac{1}{N}\sum_{j=1}^{N}\sum_{\mu\neq\nu} \xi_i^\mu \xi_j^\mu \xi_j^\nu = 1 - \xi_i^\nu h_i^\nu \tag{8}$$

where the last equality is obtained via the following steps:

$$-\xi_i^\nu A = -\xi_i^\nu \sum_j \sum_{\mu\neq\nu} \xi_i^\mu \xi_j^\mu \xi_j^\nu = -\xi_i^\nu \frac{1}{N} \sum_{j=1}^{N} (\sum_{\mu\neq\nu} \xi_i^\mu \xi_j^\mu \xi_j^\nu \underbrace{+\xi_i^\nu \xi_j^\nu \xi_j^\nu - \xi_i^\nu \xi_j^\nu \xi_j^\nu}_{\text{we add and subtract the same term}}) =$$

$$= -\xi_i^\nu \frac{1}{N} \sum_{j=1}^{N} (\sum_{\mu=1}^{p} \xi_i^\mu \xi_j^\mu \xi_j^\nu - \xi_i^\nu) = -\xi_i^\nu \sum_{j=1}^{N} \underbrace{\frac{1}{N} \sum_{\mu=1}^{p} \xi_i^\mu \xi_j^\mu}_{w_{ij}} \xi_j^\nu - \xi_i^\nu \frac{1}{N} \sum_{j=1}^{N} (-\xi_i^\nu) =$$

$$= -\xi_i^\nu h_i^\nu - \xi_i^\nu \frac{1}{N} N \xi_i^\nu = 1 - \xi_i^\nu h_i^\nu.$$

Now, if

$$\begin{cases} C_i^\nu > 1 & \text{then } 1 - \xi_i^\nu h_i^\nu > 1 \Rightarrow \xi_i^\nu h_i^\nu < 0 \Rightarrow \xi_i^\nu \text{ is not stable,} \\ C_i^\nu \leq 1 & \text{then } 1 - \xi_i^\nu h_i^\nu \leq 1 \Rightarrow \xi_i^\nu h_i^\nu \geq 0 \Rightarrow \xi_i^\nu \text{ is stable.} \end{cases}$$

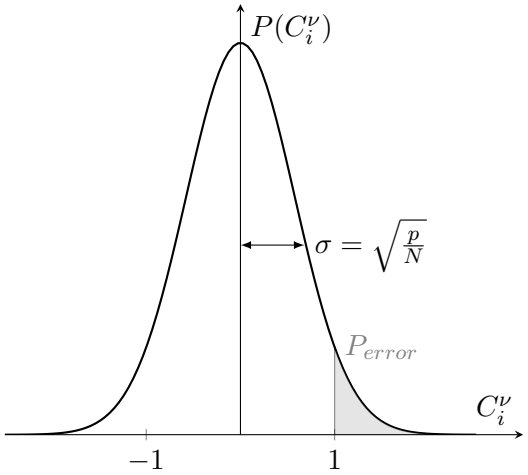
Let's hence estimate the probability P_{error} that any chosen bit is unstable:

$$P_{error} = \text{Prob}(C_i^\nu > 1). \tag{9}$$

P_{error} increases as the number of patterns p increases (the more patterns we store, the more similarity we risk).
For acceptable performance, we request $P_{error} < 0.01$.

2.5.5 Storage Capacity of the Hopfield Network

Let's find the **Storage Capacity** P_{max} of the Hopfield Network.
Let's start from the assumption that $N, p >>> 1$.



From Equation 8, we can see that $C_i^\nu = \frac{1}{N} \cdot$ (sum of $\sim N \cdot p$ random numbers (± 1))

$\downarrow$

Binomial distribution with mean 0 and variance $\sigma^2 = \frac{p}{N}$

$\downarrow$

Can be approximated with Gaussian distribution with same mean
and variance $P_{error} = \frac{1}{\sqrt{2\pi}\sigma} \int_1^\infty e^{-\frac{x^2}{2\sigma^2}} dx$

Error	p_{max}/N
0.001	0.105
0.0036	0.138
0.01	0.185
0.05	0.37
0.1	0.61

If we choose the criterion $p_{error} < 0.01 \Rightarrow p_{max} = 0.15$ N
Example: 100 units $\Rightarrow$ max capacity = 15 patterns

2.5.6 Energy Function

Another major contribution of Hopfield is to intriduce the energy function:

$$H = -\frac{1}{2}\sum_{ij}^{N} w_{ij}x_i x_j = -\frac{1}{2}\vec{x}^\top W\vec{x} \tag{10}$$

This represents an **Energy Landscape** above the configuration space (quite hilly surface).

Central Property: H always decreases (or remains constant) as the system evolves.
$\Rightarrow$ The attractors are local minima of H.

Proof H can be rewritten as $H = C - \sum_{(ij)} w_{ij}x_i x_j$, where

- C is a constant given by the ii terms:

$$\sum_{i=j} w_{ij}x_i x_j = \sum_{i=1}^{N} w_{ii}\underbrace{x_i x_i}_{=1} = \sum_{i=1}^{N}\frac{1}{N}\sum_{\mu=1}^{p}\underbrace{\xi_i^\mu \xi_i^\mu}_{=1} = \sum_{i=1}^{N}\frac{p}{N} = \text{constant}$$

- (ij) are distinct pairs, meaning that $ij = ji$ is counted only once. In fact, given the symmetries in the HN, $w_{ij}x_i x_j = w_{ji}x_j x_i$, meaning that we'd have couples of equal terms in the sum. Hence, $\sum_{ij} w_{ij}x_i x_j = 2\sum_{(ij)} w_{ij}x_i x_j$, and the 2 cancels out with the $\frac{1}{2}$ in Equation 10.

We show that the update rule $O_i(t+1) = \text{sgn}\left(\sum_j w_{ij}O_j(t)\right)$ can only decrease H.
Let's consider two cases:

1. If $O_i(t+1) = O_i(t) \Rightarrow$ the energy is unchanged.
2. If $O_i(t+1) = -O_i(t)$, then the difference in the energy function (given by the contribution of the terms involving node i):

$$
\begin{aligned}
H(t+1) - H(t) &= C - \sum_{j \neq i} w_{ij} \underbrace{O_i(t+1)}_{=-O_i(t)} \underbrace{O_j(t+1)}_{=O_j(t)} - C + \sum_{j \neq i} w_{ij} O_i(t) O_j(t) = \\
&= + \sum_{j \neq i} w_{ij} O_i(t) O_j(t) + \sum_{j \neq i} w_{ij} O_i(t) O_j(t) = \\
&= 2 O_i(t) \sum_{j \neq i} w_{ij} O_j(t) = \\
&= 2 O_i(t) \left(\sum_{j \neq i} w_{ij} O_j(t) + w_{ii} O_i(t) - w_{ii} O_i(t) \right) = \\
&= 2 O_i(t) \sum_{j} w_{ij} O_j(t) - 2 O_i(t) w_{ii} O_i(t) = \\
&= \underbrace{2 O_i(t) \sum_{j} w_{ij} O_j(t)}_{<0} - \underbrace{2 w_{ii}}_{>0} < 0 \Rightarrow H(t+1) < H(t),
\end{aligned}
$$

which means that H decreases.

$\Rightarrow$ **H decreases any time a unit x_i changes.**

$\Rightarrow$ Stable states are the minimizers of the energy function!

In the equation chain above, we used the following:

- First equal sign: definition of the rewritten H at the two time steps $t+1$ and t;
- Second: we are in CASE 2, hence $O_i(t+1) = -O_i(t)$. Also, since we are performing asynchronous updates and focusing on the change in energy due to flipping the state of node i , the state of node j remains unchanged. This implies that $O_j(t+1) = O_j(t)$.
- Third: we have the same quantity twice. We sum them and extract $O_i(t)$ from the sum as it does not depend on j, index of the sum;
- Fourth: we sum and subtract the same quantity $w_{ii} O_i(t)$ to complete the sum with the missing term for $j = i$;
- Fifth: we distribute the factor $2O_i(t)$;
- Sixth: we observe that (1) the first term is negative because $\sum_j w_{ij} O_j(t)$ is the net input, hence has the same sign of $O_i(t+1)$, which has opposite sign to $O_i(t)$, because we are in CASE 2, and (2) the second term is a positive constant ($= p/N$) as shown in the first bullet point of the proof.

3 Simple Perceptron

3.1 Proof of Convergence of the Learning Rule

Let's first define $M(\vec{w}) = \underbrace{\frac{1}{\|\vec{w}\|}}_{\text{Added so that it is a function of the direction only}} \min_{p=1\ldots P} \vec{w} \cdot \vec{\hat{x}}^p$, that is the distance of the "worst" $\vec{\hat{x}}^p$ to the plane $\perp$ to $\vec{w}$ (= MARGIN).

The quantity $M(w)$ depends on the worst of the projections and tells us how hard the problem is.

$M(\vec{w}) > 0 \Rightarrow$ all points are correctly classified
$M_{\max} = \max_w M(\vec{w})$ give us the best direction for $\vec{w} \Rightarrow$ OPTIMAL PERCEPTRON

- the larger the easier to solve the problem
- $<0 \Rightarrow$ problem cannot be solved

(Notation: we drop arrows for vectors and the hat "^" for patterns)

Proof

Let's assume that $\underbrace{\exists w^* \text{ solution to the problem}}_{M(\vec{w})>0}$ and **we proof** that it can be reached in a finite number of steps.

At each iteration of the learning process we check all the misclassified patterns ($w \cdot x^p \leq N\rho$) and use them to update w.

At some point in the learning process, we might have used the same pattern several times. Starting from the assumption that $\vec{w} = \vec{0}$ at the beginning, at this point we have

$$w = \eta \sum_{p=1}^{P} \underbrace{H^p}_{\text{times } x^p \text{ has been used}} x^p$$

Let $H = \sum_P H^P$, we want to show that H is bounded from above.

Let's consider the overlap $w \cdot w^*$

$$\begin{aligned} w \cdot w^* = \eta \sum_p H^p x_p \cdot w^* \geq \eta \sum_p H_p \min_p (x_p \cdot w^*) = \eta \min_p (x_p \cdot w^*) \sum_p H_p \\ = \eta H M(w^*) \cdot \eta \|w^*\| \end{aligned}$$

$\Rightarrow w \cdot w^*$ grows like H.

Let's now consider the change in length of w at a single update by pattern Δ

$$\begin{aligned} \Delta\|w\|^2 = \|w_{new}\|^2 - \|w\|^2 = \|w + \eta x^\nu\|^2 - \|w\|^2 = \cancel{\|w\|^2} + \eta^2\|x^\nu\|^2 + 2\eta w \cdot x^\nu - \cancel{\|w\|^2} = \\ = \eta^2 \|x^\nu\|^2 + 2\eta w \cdot x^v \leq \\ \leq \eta^2 N + 2\eta N\rho = N\eta(\eta + 2\rho) \end{aligned}$$

In the equation chain above, we assume that $x_k^\nu = \pm 1$ (but it would be anyway a finite real number for any given pattern x_k^ν) and $w \cdot x^\nu \leq N\rho$ because the pattern x_k^ν was wrongly classified. Summing these increments for H steps, we get $\|w\|^2 \leq HN\eta(\eta + 2\rho)$

$$\Rightarrow \frac{w \cdot w^*}{\|w\|} \geq \frac{\eta\sqrt{H} M(w) \cdot \|w^*\|}{\sqrt{N\eta(\eta + 2\rho)}}$$

$\Rightarrow \|w\|$ grows no faster than $\sqrt{H}$
$\Rightarrow$ grows at least as fast as $\sqrt{H}$

The first term must stop growing, because $\exists w^*$ solution $\Rightarrow H$ must stop growing too.

Let's see that:

We can bound the normalized scalar product $\phi = \frac{(w \cdot w^*)^2}{\|w\|^2 \|w^*\|^2} = \cos^2(\alpha)$

$$\Rightarrow 1 \geq \phi = \frac{(w \cdot w^*)^2}{\|w\|^2 \|w^*\|^2} \geq \frac{\eta H M(w^*)^2 \cdot \|w^*\|^2}{HN\eta(\eta + 2\rho) \cdot \|w^*\|^2} = H \frac{M(w^*)^2 \eta}{N(\eta + 2\rho)}$$

$$\Rightarrow H \leq N \cdot \frac{\eta + 2\rho}{M(w^*)^2 \eta} = N \cdot \frac{1 + 2\rho/\eta}{M_{\max}^2} \quad \square$$

We are considering the best possible solution w^* (i.e., $M(w^*) = M_{max}$)

4 Backpropagation

A network with just 1 hidden layer can represent any boolean function (including XOR)
Back-propagation ⇒ method to learn for feed-forward networks.
Back-propagation gives a prescription for changing the weights w_{pq} to learn a training set of input-output pairs (x_k^p, y_k^p). Basis: gradient descent

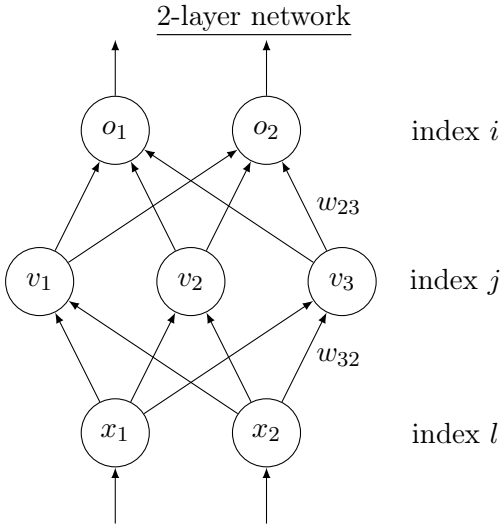
Output unit o_i
Hidden unit v_j
Input terminal x_l

$w_{jl} \rightarrow$ weight from input unit l to hidden unit j
$w_{ij} \rightarrow$ weight from hidden unit j to output unit i

$p =$ superscript for a pattern

$x_l^p \leftarrow$ input l is set to x_l^p when x^p is presented to the network; can be binary or continuous-valued

$N = \#$ input units $\quad p = \#$ inputs patterns $\quad n_k = \#$ nodes in layer k



Given a pattern p, hidden unit j receives a net input

$$h_j^\phi = \sum_l w_{jl} x_l^p$$

$\rightarrow$ output of the hidden unit $v_j^p = g\left(h_j^p\right) = g\left(\sum_l w_{jl} x_l^p\right)$
Output unit i receives $h_i^p = \sum_j w_{ij} v_j^p = \sum_j w_{ij}\left(\sum_l w_{jl} x_l^p\right)$
$\rightarrow$ output of the unit $o_i^p = g\left(h_i^p\right) = g\left(\sum_j w_{ij}\left(\sum_l w_{jl} x_l^p\right)\right)$

Cost Function $L(\vec{w}) = \frac{1}{2}\sum_{p,i}\left(y_i^p - o_i^p\right)^2$

$\xrightarrow[\text{becomes}]{}$ $L(\vec{w}) = \frac{1}{2}\sum_{p,i}\left[y_i^p - g\left(\sum_j w_{ij}\left(\sum_l w_{jl} x_l^p\right)\right)\right]^2$

continuous, differentiable function of the weights ⇒ we use gradient descent

4.1 Hidden-to-Output Connections

Gradient descent rule gives

$$\Delta w_{ij} = -\eta \frac{\partial L}{\partial w_i j} = \eta \sum_p \underbrace{\left[y_i^p - O_i^p\right] \cdot g'\left(h_i^p\right)}_{\delta_i^p} \cdot v_j^p] \qquad = \eta \sum_p \delta_i^p v_j^p$$

$\uparrow$ with $\delta_i^p = g'(h_i^p)(y_i^p - o_i^p)$

everything that does
not depend on w_{ij} disappears
when computing the derivative

4.2 Input-to-Hidden Connections

We do the same for w_{jl} using the chain rule:

$$\begin{aligned}
\Delta w_{jl} = -\eta \frac{\partial L}{\partial w_{jl}} &= -\eta \; \tfrac{\partial L}{\partial v_j^p} \cdot \tfrac{\partial v_J^p}{\partial w_{J\ell}} = \\
&= -\eta \; \tfrac{1}{2} \cdot 2 \textstyle\sum_{p,i} [y_i^p - o_i^p] \cdot [-g'(h_i^p) \cdot w_{ij}] \cdot g'(h_j^p) x_l^p \\
&= \eta \sum_{p,i} [y_i^p - o_i^p] \, g'\left(h_i^p\right) \cdot w_{ij} \cdot g'\left(h_j^p\right) x_l^p \\
&= \eta \sum_{p,i} \delta_i^p w_{ij} \cdot g'\left(h_j^p\right) x_l^p \\
&= \eta \sum_{p} \delta_j^p x_l^p \qquad\qquad \text{with } \delta_j^p = g'\left(h_j^p\right) \sum_i w_{ij} \delta_i^p
\end{aligned}$$

For the 2 types of connections we got to the same form (just a different definition of the δ).
$\Rightarrow$ for a general number of layers we always have

$$\Delta w_{pq} = \sum_{\text{patterns}} \delta_{\text{output}} \times v_{\text{input}}$$

The term δ_{output} is defined in terms of the δ's of the units that it feeds (for example, above, δ_j^p for the hidden unit v_j^p depends on δ_i^p for the output units o_i that is feeds)
The coefficient w_{ij} propagate the errors (δ's) backwards instead of signal forward $\Rightarrow$ Hence the name **Back-Propagation**
All the seen formulas are defined with sums over the patterns. However, usually, only one pattern at a time is used to update the weight (this is convenient with redundant datasets)

4.3 Summary - A step-by-step procedure

Labeling

- $K = \#$ layers K (index)
- $v_i^k =$ output of the i-th unit of layer K
- ($v_i^0 = x_i$ and $v_i^K = O_i$) superscript now is for the layer and not for the pattern (we consider using only one pattern at a time)
- $w_{ij}^k =$ connection from v_j^{k-1} to v_i^k

Back-Propagation Procedure

1. Initialize the weights to small random values
2. Choose a pattern x_l^p and apply it to the input layer ($k = 0$) so that $v_i^0 = x_l^p \quad \forall l$
3. Propagate the signal forward: $v_i^k = g(h_i^k) = g(\sum_j w_{ij}^k v_j^{k-1})$ until the final outputs v_i^k are computed
4. Complete deltas for last layer: $\delta_i^k = g'(h_i^k)[y_i^p - v_i^k]$
5. Compute deltas for preceding layers by propagating errors backwards $\delta_i^{k-1} = g'(h_i^{k-1}) \sum_j w_{ji} \delta_j^k$ for $k = k, k-1, \ldots, 2$
6. Use $\Delta w_{ij}^k = \eta \delta_i^k v_j^{k-1}$ to update all connections according to $\Delta w_{ij}^k = w_{ij} + \Delta w_{ij}$
7. Got to step (2) and repeat for next pattern

A Derivatives

A.1 Derivatives of a Sum/Product/Chain Rule

$f(x) = 3x^2$
$g(x) = \log x$
$(f(x) + g(x))' = (3x^2 + \log x)' = 6x + \frac{1}{x} = \frac{6x^2+1}{x}$
$(f(x) \cdot g(x))' = (3x^2 \cdot \log x)' = 6x \log x + 3x^2 \frac{1}{x} = 3x(2 \log x + 1)$
$[g(f(x))]' = g'(f(x)) \cdot f'(x) = \frac{1}{3x^2} \cdot 6x = \frac{2}{x}$

A.2 For multivariate Functions $g(f(x, y))$

$g(f(\vec{x})) = e^{x_1 x_2^2} \qquad f(\vec{x}) = x_1 x_2^2 \quad g(y) = e^y$
$(x_1, x_2) \longrightarrow \underbrace{f(x_1, x_2)}_{x_1 x_2^2} \longrightarrow \underbrace{g(f(x_1, x_2))}_{e^{x_1 x_2^2}} = 0$
$\frac{\partial O}{\partial x_1} = e^{x_1 x_2^2} \cdot x_2^2 \quad \frac{\partial O}{\partial x_2} = e^{x_1 x_2^2} \cdot 2x_1 x_2 = 2x_1 x_2 \cdot e^{x_1 x_2^2}$

A.3 One level up: multivariable chain rule $g(f_1(x), f_2(x))$

$z = f(x, y)$ where x and y depend on one or more variables

$$\hookrightarrow x = x(t) \quad \text{and} \quad y = y(t)$$

$z = x^2 y - y^2 \qquad x = t^2 \quad y = 2t$

$$\begin{aligned}\frac{dz}{dt} &= \frac{\partial z}{\partial x}\frac{dx}{dt} + \frac{\partial z}{\partial y}\frac{dy}{dt} = \\ &= (2xy)(2t) + (x^2 - 2y)2 = (2t^2 \cdot 2t)(2t) + 2(t^4 - 4t) \\ &= 8t^4 + 2t^4 - 8t = 10t^4 - 8t\end{aligned}$$

If $x = x(u, v)$ and $y = y(u, v)$ and $z = f(x, y) \Rightarrow z(u, v) = f(x(u, v), y(u, v))$ has first-order partial derivatives at (u, v) given by

$$\frac{\partial z}{\partial u} = \frac{\partial z}{\partial x}\frac{\partial x}{\partial u} + \frac{\partial z}{\partial y}\frac{\partial y}{\partial u}$$

$$\frac{\partial z}{\partial v} = \frac{\partial z}{\partial x}\frac{\partial x}{\partial v} + \frac{\partial z}{\partial y}\frac{\partial y}{\partial v}$$

Example $z = e^{x^2 y} \quad x(u, v) = \sqrt{uv} \quad y(u, v) = \frac{1}{v}$

$$\begin{aligned}\frac{\partial z}{\partial u} &= e^{x^2 y} \cdot 2xy \cdot \frac{1}{2\sqrt{uv}} \cdot v + \cancel{e^{x^2 y} \cdot x^2 \cdot 0} \\ &= e^{u\cancel{v} \cdot \frac{1}{\cancel{v}}} \cdot \cancel{2\sqrt{uv}} \cdot \frac{1}{\cancel{v}} \cdot \frac{1}{\cancel{2\sqrt{uv}}} \cdot \cancel{v} = e^u\end{aligned}$$

$$\begin{aligned}\frac{\partial z}{\partial v} &= e^{x^2 y} \cdot 2xy \cdot \frac{1}{2\sqrt{uv}} \cdot u + e^{x^2 y} \cdot x^2 \cdot \left(-\frac{1}{v^2}\right) \\ &= e^u \cdot \cancel{2\sqrt{uv}} \cdot \frac{1}{v} \cdot \frac{1}{\cancel{2\sqrt{uv}}} \cdot u + e^u \cdot u\cancel{v}\left(-\frac{1}{v^2}\right) \\ &= \frac{u}{v}e^u - \frac{u}{v}e^u = 0\end{aligned}$$